

Open Window

MVP Product Pack and Codex Delivery Guide

Working title: Open Window

Status: Proposed starting point

Owner: Founder / product owner

Version: 0.1

Date: 18 July 2026

Product promise: A calm, private companion for fasting, food, hydration and progress that remains useful even when the user forgets it for a few days.

Open Window begins with a personal result: a simple daily loop helped its founder lose 27 kg. The app will be shared freely so other people can try the same mechanics for themselves. It will not have subscriptions, advertising, premium gates or shame-based streaks.

This pack converts the existing product-vision storyboard into a buildable MVP. It records the product decisions already made, exposes the assumptions still requiring validation, defines the first backlog, and provides a repeatable format for giving work to Codex.

Contents

- **Product contract:** 1. Executive decision · 2. Why this product should exist · 3. Product principles · 4. Primary user · 5. MVP boundary.
- **Experience definition:** 6. Information architecture · 7. Core journeys · 8. Domain language and rules · 9. Data model · 10. Measures and guardrails.
- **Delivery system:** 11. Story map and backlog · 12. Build sequence · 13. Definition of Ready · 14. Definition of Done · 15. Decisions.
- **Codex handoff:** 16. Story template · 17. Ready examples · 18. Codex prompt template.
- **Recommended path:** 19. Product-management path · 20. Solo cadence · 21. Sources.
- **Appendices:** A. Readiness checklist · B. Immediate next session.

1. Executive decision

Recommendation

Start coding after a short readiness pass, not after writing a complete specification for the whole app. Confirm the handful of product and technical decisions listed in section 15, set up the repository and test harness, then build the MVP as six narrow, usable vertical slices.

The first slice should prove the core loop: choose a fasting goal, start a fast, see its progress and end it. Photo recognition and AI nutrition interpretation are deliberately deferred until the manual mechanics are trustworthy.

MVP in one sentence

A user can set a fasting goal of at least 12 hours; record and correct fasts, food and hydration; return after missing days and reconstruct an honest history; and view Apple Health weight and movement alongside their record.

Product goal

Enable a self-directed iPhone user to understand and maintain the shape of their eating, drinking and fasting week in less than a minute a day, without punishment for gaps.

Release hypothesis

If the app makes the current fasting state obvious, makes manual logging quick, and makes gaps recoverable, then users will return to it because it reduces uncertainty rather than creating another obligation.

2. Why this product should exist

The user problem

People trying to change their weight or eating routine often have several disconnected records: food in one place, weight in Apple Health, steps somewhere else, and fasting in a timer. Many products add content, coaching, subscriptions and streak pressure. When real life interrupts logging, the incomplete record makes the product less trustworthy and returning feels like failure.

The opportunity

Keep the useful behavioural loop while removing commercial noise:

- Make today's state immediately legible.
- Keep the number of daily decisions low.
- Treat missed days as normal.
- Make estimates and unknowns visible.
- Put the user's health data to work only for that user.
- Remain free, private and calm.

Jobs to be done

- When I am between meals, help me see how long I have been fasting and what my goal means today.
- When I eat or drink, let me record it quickly enough that logging does not become the work.
- When I return after a gap, help me repair what I remember without inventing what I do not.
- When I review progress, combine my record with Apple Health weight and movement so I can see trends rather than judge one day.

3. Product principles

Calm over compulsion

The app may encourage clarity and continuation, but it must not create anxiety through streak loss, red warnings or guilt. Neutral language is a functional requirement.

Honest over impressive

The interface distinguishes recorded facts, user-confirmed reconstructions and unknown periods. It must not present speculative physiological stages, nutrition estimates or reconstructed facts as measured facts.

Useful after absence

A user who has not opened the app for several days is a normal user, not an exception. Catch-up is a first-class journey and a defining feature of the MVP.

Private by default

Prefer on-device storage. Request Apple Health access only in context, ask only for data required by the feature, and do not use health data for advertising, marketing or unrelated analytics.

Free without a catch

No subscription, premium tier, advertising, trials, feature gates or manipulative upgrade surfaces.

One obvious next action

Every primary screen should make the user's current state and the next reasonable action clear.

4. Primary user and use context

Primary user

A busy, self-directed adult who wants a lightweight structure for weight-management habits. They already record some weight or movement data in Apple Health, may use a time-restricted eating routine, and will sometimes forget to log for several days.

Behavioural characteristics

- Values simplicity more than coaching content.
- Wants a useful record but not a perfect record.
- Can understand and control their own fasting goal.
- Needs fast correction and backdating.
- Is sensitive to judgemental language and false precision.

Initial research cohort

The founder-user is the first research participant, not proof of a general market. Before public release, observe three to five other people attempting the core tasks and include at least one person who intentionally returns after a simulated three-day gap.

Not designed for

The MVP is not a medical device, diagnostic service, prescribed treatment, eating-disorder intervention, child-focused product or personalised nutrition programme. Health and fasting copy should advise users to seek appropriate professional guidance when fasting may be unsuitable for them.

5. The MVP boundary

In scope

Onboarding and settings

- Explain the product promise and non-medical nature in plain language.
- Choose a fasting goal, with 12 hours as the minimum and starting default.
- Change the goal later without rewriting completed history.
- Choose hydration quick-add amounts and basic display preferences.

Fasting

- Start a fast now or at a chosen past time.
- Show elapsed time, target time and current status.
- End a fast now or at a chosen past time.
- Edit or delete an incorrect fast.
- View fasting history.
- Show an active fast on the Lock Screen and Dynamic Island using Live Activities where supported.

Food

- Add a food event using a short text description and time.
- Optionally enter known nutrition values manually.
- Edit, backdate or delete an entry.
- Mark an entry as containing calories for fasting-history purposes.

Hydration

- Quick-add water, tea, coffee and a custom drink.
- Record amount, unit and time.
- Edit, backdate or delete an entry.
- Let the user mark whether a drink contains calories; hydration does not automatically mean a fast was broken.

Catch-up and reconstruction

- Open any recent past day and add remembered food or hydration.
- Identify possible fasting periods from confirmed caloric events.

- Preview proposed fasts before saving them.
- Label reconstructed fasts as user-confirmed.
- Preserve unknown periods when evidence conflicts or is missing.
- Recalculate only affected history when a past entry changes.

Progress and Apple Health

- Request read access to weight and step count only when the user opens or enables progress features.
- Show recent weight and a simple trend from Apple Health.
- Show daily steps or movement context from Apple Health.
- Handle denied, limited, unavailable and later-revoked access gracefully.
- Treat Apple Health as the source of truth for these samples in the MVP.

Data control and quality

- Work offline for all manual logging and fasting functions.
- Store core app data on device.
- Allow the user to delete app-created local data.
- Support Dynamic Type, VoiceOver labels, sufficient contrast and non-colour state cues.
- Provide a clear privacy policy and health disclaimer before App Store submission.

Explicitly out of scope for MVP

- Food-photo capture or recognition.
- AI interpretation of food descriptions.
- Automatic macro or micronutrient calculation.
- Personalised coaching, recommendations or chat.
- Claims about blood sugar, ketosis, fat burning or other biological states.
- Accounts, sign-in, cloud sync or multi-device merge.
- Social feeds, leaderboards or public profiles.
- Subscription, advertising, donations inside the core experience or premium gates.
- Apple Watch app.
- HealthKit write access.
- Barcode scanning, recipe import or restaurant database.
- Calorie targets or medical recommendations.

Feature 1 after MVP

Feature 1 can add photo capture and AI-assisted food interpretation once manual food entry, correction, history and data trust are working. AI output must be presented as an estimate, remain editable, state uncertainty, and never silently overwrite the user's record.

6. Information architecture

Use four primary destinations and avoid a deep menu hierarchy.

Today

Current fast state, one primary fasting action, today's food and hydration summary, and quick-add actions.

History

Calendar or day list, past entries, completed and reconstructed fasts, correction and catch-up entry points.

Progress

Weight trend and steps from Apple Health, plus a neutral weekly reflection.

Settings

Fasting goal, hydration defaults, Health permissions status, privacy, data deletion, accessibility preferences and about text.

7. Core journeys

Journey A: First use and first fast

1. User reads the short product promise.
2. User chooses a goal of 12 hours or more.
3. User arrives on Today with no fast active.
4. User starts a fast now or corrects its start time.
5. Today and the Live Activity show elapsed time and the goal end time.
6. User ends the fast and sees it in History.

Success: the user can complete this without understanding fasting jargon or opening Settings.

Journey B: Record today's food and hydration

1. User chooses Food or a hydration quick action.
2. User records a description or drink type, amount and time.
3. The entry appears immediately in today's timeline.
4. User can undo or edit it without hunting through menus.
5. A caloric entry can end an active fast only after the intended behaviour is clear to the user.

Success: common hydration takes two taps after opening the app; a basic food event takes less than 20 seconds.

Journey C: Return after missing days

1. App welcomes the user back without a streak warning.
2. User selects Catch up or opens a past day.
3. User adds remembered eating and drinking events with past timestamps.
4. App proposes possible fasts between confirmed caloric events.
5. User reviews, edits and confirms proposals individually or as a clear batch.
6. Ambiguous periods remain unknown.

Success: the user can repair three days of remembered history in under three minutes and understands which fasts were reconstructed.

Journey D: Connect Apple Health and review progress

1. User opens Progress and sees why weight and step access would help.
2. The system Health permission flow appears in context.
3. On approval, recent weight and steps appear with source and recency.
4. On denial or absence of data, the screen remains useful and explains the next option without repeatedly prompting.

Success: the user understands what is read, why it is read and how to change permission later.

8. Domain language and rules

Core terms

Fast: a time interval recorded by the user or reconstructed and confirmed by the user. It is not proof of a biological state.

Active fast: a fast with a start time and no end time.

Goal: the user's target duration for future and active fasts. Minimum 12 hours.

Food event: a timestamped description of eating, optionally with manually supplied nutrition values.

Hydration event: a timestamped drink type and amount, optionally marked as caloric.

Caloric event: a food or drink event that the user intends to count as ending a fast.

Recorded fast: explicitly started or entered by the user.

Reconstructed fast: derived from confirmed caloric event times and saved only after user confirmation.

Unknown period: time for which the app lacks enough trusted information to assert a fast.

Business rules

- BR-01: A fasting goal cannot be set below 12 hours.
- BR-02: Changing the goal affects active and future presentation; it does not alter completed fast durations.
- BR-03: Only one active fast may exist at a time.
- BR-04: Start and end times may be backdated but an end must be later than its start.
- BR-05: A completed fast records the goal that applied while it was active for historical context.
- BR-06: A hydration event does not break a fast unless it is marked caloric.
- BR-07: A food event is caloric by default, but the user can correct it.
- BR-08: Adding a caloric event inside an active fast prompts the user to end or correct the fast; the app does not silently change it.
- BR-09: Reconstruction uses confirmed caloric events as boundaries and never saves a proposed fast without user confirmation.
- BR-10: Conflicting or insufficient evidence produces an unknown period, not an invented fast.
- BR-11: Editing or deleting a past boundary invalidates affected reconstructed fasts and prompts for review.
- BR-12: Time-zone and daylight-saving changes preserve the stored instants and display understandable local times.
- BR-13: Apple Health weight and steps are read-only in the MVP and remain attributable to HealthKit.

- BR-14: Permission denial never blocks manual logging or fasting.
- BR-15: User-facing health copy describes records and patterns, not diagnoses or guaranteed physiological effects.

9. Conceptual data model

The implementation may evolve, but stories should use consistent product concepts.

- **UserSettings:** fastingGoalHours, hydrationDefaults, displayUnit, onboardingState.
- **FastRecord:** id, startAt, endAt, goalHoursAtStart, origin, confirmationState, createdAt, updatedAt.
- **FoodEntry:** id, occurredAt, description, isCaloric, optionalNutrition, createdAt, updatedAt.
- **HydrationEntry:** id, occurredAt, drinkType, amount, unit, isCaloric, createdAt, updatedAt.
- **ReconstructionProposal:** id, proposedStartAt, proposedEndAt, boundaryEntryIds, status, explanation.
- **HealthSnapshot:** HealthKit identifiers or query anchors sufficient to display weight and step results without duplicating them as app-owned truth.

Data-state labels

Use three visible provenance states where relevant:

- Recorded by you.
- Reconstructed and confirmed by you.
- Unknown.

10. Success measures and guardrails

MVP outcome measures

Measure these initially through structured founder testing and a small opt-in test cohort, not invasive analytics.

- **First-value success:** a new user sets a goal and records a first fast without assistance.
- **Daily effort:** hydration quick add takes no more than two taps after launch; basic food logging takes under 20 seconds.
- **State comprehension:** users can accurately explain whether they are fasting, how long, and when the goal is reached.

- **Recovery success:** users can reconstruct three missed days and distinguish confirmed from unknown history.
- **Correction success:** users can find and fix an incorrect timestamp without assistance.
- **Health trust:** users can explain what Apple Health data is read and why.
- **Reliability:** no lost or duplicated core records across normal relaunch, backgrounding or time-zone change tests.

Guardrails

- Do not optimise for streak length or maximum fasting duration.
- Do not use fear, guilt or body-shaming language.
- Do not claim health outcomes from app usage.
- Do not collect health data for advertising or unrelated profiling.
- Do not hide unknown or reconstructed data behind the same visual treatment as directly recorded data.

11. Story map and initial backlog

Priority uses **P0** for required MVP, **P1** for MVP polish that may follow the first internal build, and **Later** for post-MVP.

Epic E0: Product foundation

- **OW-000 · P0 · Technical enabler:** Create an iPhone project shell with four destinations, local persistence, test targets and repository guidance.
- **OW-001 · P0:** Understand the promise and complete minimal onboarding.
- **OW-002 · P0:** Set and later change a fasting goal of at least 12 hours.
- **OW-003 · P1:** Delete all app-created local data from Settings.

Epic E1: Fasting loop

- **OW-101 · P0:** Start a fast now.
- **OW-102 · P0:** Start or correct a fast using a past time.
- **OW-103 · P0:** See elapsed time, goal progress and target time.
- **OW-104 · P0:** End an active fast now or at a past time.
- **OW-105 · P0:** View, edit and delete completed fasts.
- **OW-106 · P1:** See the active fast in a Lock Screen Live Activity.

Epic E2: Manual daily log

- **OW-201 · P0:** Add, edit, backdate and delete a text food event.
- **OW-202 · P0:** Mark whether a food or drink event counts as caloric.
- **OW-203 · P0:** Quick-add water, tea or coffee.
- **OW-204 · P0:** Add and edit a custom hydration event.
- **OW-205 · P0:** View today's food and hydration in one timeline.
- **OW-206 · P1:** Undo the most recent logging action from Today.

Epic E3: Catch-up and history repair

- **OW-301 · P0:** Open a recent past day and add entries with historical timestamps.
- **OW-302 · P0:** Generate fasting proposals between confirmed caloric boundaries.
- **OW-303 · P0:** Preview, edit, accept or reject each proposed fast.
- **OW-304 · P0:** Label reconstructed fasts and preserve unknown periods.
- **OW-305 · P0:** Re-evaluate affected proposals when a boundary entry changes.

Epic E4: Apple Health and progress

- **OW-401 · P0:** Explain and request read access to weight in context.
- **OW-402 · P0:** Display recent HealthKit weight and a simple trend.
- **OW-403 · P0:** Explain and request read access to step count in context.
- **OW-404 · P0:** Display daily steps with source and recency.
- **OW-405 · P0:** Handle unavailable, denied and revoked permissions gracefully.

Epic E5: Release quality

- **OW-501 · P0:** Complete VoiceOver, Dynamic Type, contrast and reduced-motion checks for core journeys.
- **OW-502 · P0:** Add privacy policy, permission-purpose text and non-medical disclaimer.
- **OW-503 · P0:** Verify offline persistence, migration safety and recovery after interrupted writes.
- **OW-504 · P1:** Create an opt-in shareable progress summary without exposing raw health data by default.

Post-MVP Epic F1: Assisted food capture

- **OW-F101 · Later:** Capture a food photograph.
- **OW-F102 · Later:** Generate an editable estimated description and nutrition breakdown.

- **OW-F103 · Later:** Show confidence and the basis of estimates.
- **OW-F104 · Later:** Learn reusable meals locally with user control.

12. Recommended build sequence

Build usable vertical slices rather than completing every data layer before showing a screen.

Slice 0: Foundation and decisions

Goal: a repository Codex can work in safely and consistently.

- Confirm the decisions in section 15.
- Create the Xcode project, navigation shell and on-device persistence approach.
- Add unit and UI test targets.
- Add repository-level AGENTS.md, build commands and Definition of Done.
- Create a small fixture set for dates, time zones and sample HealthKit states.

Slice 1: First fasting loop

Goal: choose a goal, start, observe and end a fast.

Stories: OW-001, OW-002, OW-101, OW-103 and OW-104.

Validation: complete the journey on a physical iPhone and after app relaunch.

Slice 2: Today's manual record

Goal: record food, tea, coffee and water in a useful timeline.

Stories: OW-201 through OW-205.

Validation: log a normal day and correct three intentional mistakes.

Slice 3: Catch-up

Goal: repair a three-day gap without false certainty.

Stories: OW-301 through OW-305.

Validation: test complete, incomplete and conflicting histories.

Slice 4: Apple Health progress

Goal: show weight and steps with contextual permission and robust empty states.

Stories: OW-401 through OW-405.

Validation: test granted, denied, no-data and revoked states on a device.

Slice 5: Glanceability and release quality

Goal: make the product dependable and ready for a small external test.

Stories: OW-106 and OW-501 through OW-503.

Validation: accessibility audit, interruption tests, time-zone tests, privacy review and five-person task observation.

13. Definition of Ready

A story is ready to give to Codex when all applicable checks are true:

- The user, need and outcome are clear.
- It supports the current Product Goal and has a priority.
- It is small enough to implement and verify as one coherent change.
- In-scope and out-of-scope behaviour are explicit.
- Acceptance criteria are observable and testable.
- Loading, empty, denied, error and correction states are addressed where relevant.
- Product rules and terminology are referenced by ID.
- Required design or storyboard context is linked.
- Data, privacy, HealthKit and accessibility implications are stated.
- Dependencies and migrations are known.
- No unresolved question would materially change the implementation.
- The expected automated and manual verification is specified.

If any item fails, ask Codex to help refine or spike the uncertainty before asking it to implement the feature.

14. Definition of Done

A story is done only when:

- Every acceptance criterion passes.
- Relevant unit, integration and UI tests are added or updated and pass.
- The app builds without new warnings in the supported configuration.

- Persistence and migration behaviour is safe for existing test data.
- Core behaviour has been manually exercised on a simulator and on a physical device when Apple frameworks require it.
- VoiceOver labels, Dynamic Type, contrast, focus order and non-colour cues have been checked for changed screens.
- Permission and privacy copy accurately describes the data used.
- Error, empty and denied states do not trap the user.
- The diff has been reviewed for regressions and unnecessary scope.
- Product documentation and the decision log are updated when behaviour changed.
- No photo, AI, coaching, monetisation or unrequested expansion has entered MVP scope.

15. Decisions required before Slice 1

These are deliberately few. Resolve them in one product session and record the answers.

1. **Fast start model:** manual start only for the first slice, or automatically offer a start from the latest confirmed caloric event? **Recommendation:** manual first; add a non-destructive suggestion later.
2. **Goal choices:** whole hours only or finer increments? **Recommendation:** whole hours from 12 to 24 for MVP, with 12 as default.
3. **Food nutrition in MVP:** description only or optional manual values? **Recommendation:** description plus optional manual values hidden behind “Add details.”
4. **Catch-up horizon:** how far back should the guided flow initially cover? **Recommendation:** seven days, while History still permits older manual entries.
5. **Health data:** weight and steps read-only? **Recommendation:** yes for MVP; avoid ownership and conflict complexity.
6. **Storage:** local device only? **Recommendation:** yes for MVP; do not store personal health information in iCloud.
7. **Minimum iOS version:** choose based on Live Activities support and devices available for testing. **Recommendation:** record this as an architecture decision before project creation.
8. **Working name:** continue with Open Window until naming checks are worthwhile. The name must not delay the first internal build.

16. User-story template

Use this structure for every implementation story. The story sentence keeps the user outcome visible; the remaining fields make the work safe and testable.

[ID] Short outcome title

Epic:

Priority: P0 / P1 / Later

Status: Draft / Ready / In progress / Done

User story

As a [specific user/context], I want [outcome], so that [benefit].

Why now

How this advances the current Product Goal or unlocks a journey.

Context

Relevant product-pack sections, storyboard slides, repository files and prior decisions.

In scope

- Observable behaviour included in this story.

Out of scope

- Nearby behaviour deliberately excluded.

Product rules

Reference BR identifiers and define any story-specific rule.

Acceptance criteria

- Given [starting state], when [action], then [observable result].
- Given [edge or error state], when [action], then [safe observable result].

States and edge cases

- Empty, loading, error, denied, offline, interrupted, duplicate, time-zone and accessibility states as applicable.

Data and privacy

Data read or written, retention, permission and deletion implications.

Design and content

Screen reference, hierarchy, exact critical copy and accessibility labels.

Dependencies

Stories, frameworks, decisions, fixtures or technical spikes.

Verification

- Automated tests to add or update.

- Manual scenarios and required device conditions.
- Build, lint or review commands.

Done when

Story acceptance criteria and the project Definition of Done are satisfied.

17. Codex-ready example stories

OW-002 Set a fasting goal

Epic: E0 Product foundation

Priority: P0

Status: Ready after minimum iOS and persistence decisions

User story

As a new or returning user, I want to choose a fasting goal of at least 12 hours, so that the timer reflects the routine I intend to follow.

Why now

The goal is required before the first complete fasting loop can be tested.

In scope

- Default the first-use choice to 12 hours.
- Offer whole-hour choices from 12 through 24.
- Save the choice on device.
- Allow the choice to be changed in Settings.
- Update the active fast's target presentation without changing its start time.
- Preserve completed fast durations and their historical goal.

Out of scope

- Different goals by weekday.
- Goals below 12 hours or above 24 hours.
- Coaching or recommendation of a goal.

Product rules: BR-01, BR-02, BR-05.

Acceptance criteria

- Given a first launch, when the goal step appears, then 12 hours is selected by default and Continue is available.

- Given a user selects 16 hours and continues, when the app is relaunched, then 16 hours remains selected.
- Given an active fast and a change from 16 to 14 hours, when the user returns to Today, then the target time reflects 14 hours and the start time is unchanged.
- Given a completed fast created under a 16-hour goal, when the current goal changes, then that fast retains its recorded 16-hour historical goal.
- Given any goal-control interaction, then VoiceOver announces the value and selected state and the control works at large Dynamic Type sizes.

Verification

- Unit-test minimum, maximum, persistence and historical-goal behaviour.
- UI-test first-use selection and Settings change.
- Manually check 12, 16 and 24 hours with large Dynamic Type.

OW-101 Start a fast now

Epic: E1 Fasting loop

Priority: P0

Status: Ready after OW-002

User story

As a user who has finished eating, I want to start a fast with one obvious action, so that I can see its progress without remembering the exact time.

In scope

- Show Start fast as the primary action when no fast is active.
- Use the current instant as the start time.
- Persist one active fast.
- Immediately update Today with elapsed time, goal and target time.
- Make repeated taps idempotent.

Out of scope

- Automatic start from a meal.
- Backdated start, which belongs to OW-102.
- Live Activity, which belongs to OW-106.

Product rules: BR-02, BR-03, BR-05.

Acceptance criteria

- Given no active fast, when Start fast is tapped, then one active fast is saved with a start time matching the current instant within one second.
- Given the fast was saved, when Today renders, then it shows elapsed time, the selected goal and target time.
- Given the app is terminated and relaunched, then the same active fast continues from its stored start time.
- Given an active fast already exists, when the start action is triggered again through any race or duplicate event, then no second active fast is created.
- Given persistence fails, then the UI does not pretend the fast started and offers a calm retry state.

Verification

- Unit-test creation and single-active-fast invariant using an injectable clock.
- UI-test start and relaunch.
- Test duplicate taps and simulated persistence failure.

OW-203 Quick-add hydration

Epic: E2 Manual daily log

Priority: P0

Status: Ready after the local entry model exists

User story

As a user holding a drink, I want to record water, tea or coffee in two taps, so that hydration tracking does not interrupt my day.

In scope

- Provide visible quick actions for water, tea and coffee on Today.
- Use the user's configured default amount for each type.
- Record the current time and non-caloric state by default.
- Show the new event in today's timeline and update the total.
- Provide an immediate undo affordance.

Out of scope

- Caffeine analysis.
- Automatic detection.
- HealthKit hydration write access.

Product rules: BR-06, BR-14.

Acceptance criteria

- Given Today is open, when Water and the confirmation action are tapped, then one water event is recorded with the configured amount and current time.
- Given Tea or Coffee is recorded, then it contributes to the displayed hydration total.
- Given a default quick-add drink, then it does not end an active fast.
- Given Undo is tapped before it expires, then the entry and displayed total return to the prior state.
- Given the device is offline, then quick add works normally.

Verification

- Unit-test totals, default values, undo and non-caloric behaviour.
- UI-test a two-tap water flow and VoiceOver labels.

OW-201 Add a manual food event

Epic: E2 Manual daily log

Priority: P0

Status: Ready after the local entry model exists

User story

As a user who has eaten, I want to record a short description and time, so that my eating history is useful without needing photo or AI analysis.

In scope

- Require a non-empty plain-text description.
- Default to the current time and allow a past time.
- Mark the entry caloric by default and let the user correct that choice.
- Offer optional manual nutrition details behind a secondary action.
- Add, edit and delete the event.

Out of scope

- Text interpretation, food database lookup, photo input or generated nutrition values.

Product rules: BR-07, BR-08, BR-15.

Acceptance criteria

- Given the add-food form, when a description and valid time are saved, then the event appears at the correct position in the day timeline.

- Given an empty or whitespace-only description, then Save remains unavailable and an accessible explanation is present.
- Given an active fast and a new caloric food event within it, then the app asks whether to end or correct the fast and makes no silent change.
- Given optional nutrition values are omitted, then the event saves without invented values or warnings.
- Given a food event is edited or deleted, then affected reconstruction proposals are marked for review.

Verification

- Unit-test validation, sorting, caloric default and affected-history invalidation.
- UI-test add, edit, delete and active-fast conflict.

OW-302 Propose reconstructed fasts

Epic: E3 Catch-up and history repair

Priority: P0

Status: Ready after OW-201, OW-204 and OW-301

User story

As a returning user who has entered remembered meals, I want the app to propose possible fasting periods for review, so that I can rebuild history without manually calculating every interval.

In scope

- Consider confirmed caloric food and drink events within the selected catch-up range.
- Propose an interval after one caloric event and before the next caloric event.
- Show both boundary events, duration and why the interval was proposed.
- Never save a proposal automatically.
- Allow edit, accept or reject.
- Save accepted intervals with reconstructed-and-confirmed provenance.

Out of scope

- Inferring missing meals from habits.
- Claiming the user was physiologically fasting.
- AI-based reconstruction.

Product rules: BR-09, BR-10, BR-11, BR-12.

Acceptance criteria

- Given two confirmed caloric events with no conflicting record between them, when reconstruction runs, then one proposal spans the first event to the second and cites both boundaries.
- Given a proposal, when the user accepts it, then a reconstructed-and-confirmed fast is saved and visibly labelled.
- Given a proposal, when the user rejects it, then no fast is saved and the interval remains unknown.
- Given conflicting recorded fasts or impossible timestamps, then no proposal is silently produced and the period is flagged for review.
- Given a boundary event is edited after acceptance, then the affected reconstructed fast is invalidated for review rather than silently rewritten.
- Given a daylight-saving transition, then the duration is calculated from absolute instants and local times are displayed clearly.

Verification

- Unit-test simple boundaries, overlaps, rejection, invalidation, missing evidence and daylight-saving cases.
- UI-test preview, edit, accept and provenance label.

OW-401 Connect Apple Health weight

Epic: E4 Apple Health and progress

Priority: P0

Status: Ready after HealthKit entitlement and device-test setup

User story

As a user who records weight in Apple Health, I want Open Window to read it with my permission, so that progress appears without duplicate entry.

In scope

- Explain the direct user benefit when Progress first needs weight.
- Invoke Apple's standard Health authorization flow.
- Request read access to body mass only for this story.
- Query and show the latest available sample and a simple trend.
- Handle unavailable, no-data, denied and later-revoked states.

Out of scope

- Writing weight to HealthKit.
- Cloud copy of weight.
- Medical interpretation or target-weight recommendation.

Product rules: BR-13, BR-14, BR-15.

Acceptance criteria

- Given access has not been requested, when the user chooses Connect weight in Progress, then the app explains why immediately before the system flow.
- Given access and available samples, then the most recent weight, unit, date and neutral trend are displayed.
- Given no available samples, then the app shows a useful empty state without claiming permission was denied.
- Given access is denied or later unavailable, then fasting and manual logging remain fully usable and Progress explains how system settings control access.
- Given a weight sample changes in HealthKit, then a refresh reflects the HealthKit result without creating a duplicate app-owned weight entry.

Verification

- Unit-test query mapping and view states behind a HealthKit abstraction.
- UI-test mocked granted, no-data and unavailable states.
- Manually test system authorization and revocation on a physical iPhone.

18. Prompt template for giving a ready story to Codex

Codex works best when the task states the goal, relevant context, constraints and what must be true when it is done. Use the following prompt after a story passes Definition of Ready.

Implement story **[ID and title]** from docs/BACKLOG.md.

Goal: Deliver the user outcome and all acceptance criteria for this story.

Context: Read AGENTS.md, docs/PRODUCT.md, docs/MVP_SCOPE.md, docs/DOMAIN_RULES.md, docs/DECISIONS.md and the complete story before changing code. Inspect the existing implementation and tests first.

Constraints: Keep the change within the story's in-scope boundary. Preserve local-first privacy, accessibility, the stated domain rules and existing architecture. Do not add photo, AI, coaching, cloud, monetisation or unrelated refactors. Flag a contradiction or material missing decision before implementing it.

Done when: Implement the smallest coherent solution; add or update the specified tests; run the repository build, test and lint commands; manually verify any listed device scenario that is possible in this environment; review the diff against the acceptance criteria; and summarise changed files, verification results, assumptions and remaining risks.

For a complex story, ask Codex to plan first. For a small, well-understood story, let it implement directly. Keep one story or one tightly related vertical slice per task so review remains possible.

19. Product-management path from here

Step 1: Ratify the product contract

Review sections 1 through 10 and resolve the eight decisions in section 15. Treat the pack as a living product contract, not a promise that every early assumption is correct.

Exit condition: product goal, MVP boundary, core rules and open decisions are explicit.

Step 2: Validate the risky journeys cheaply

Use the existing storyboard or a tappable prototype to test first-use fasting, today's logging, catch-up and Health permission explanation. Test behaviour, comprehension and trust; do not ask whether people “like the idea.”

The highest product risks are:

- **Value:** does a calm combined record help enough to return to?
- **Usability:** can people understand active, recorded, reconstructed and unknown states?
- **Feasibility:** can catch-up, HealthKit and Live Activities behave reliably across time and device states?
- **Viability:** can a free, privacy-first app remain supportable and App Store compliant?

Exit condition: critical confusion is corrected in the product rules or design, and no known usability problem is simply passed to coding.

Step 3: Establish the technical runway

Decide minimum iOS, SwiftUI and persistence choices; create the repository; document build/test commands; create a clock abstraction; isolate HealthKit and ActivityKit behind testable boundaries; and add fixtures for edge dates and permission states.

Exit condition: OW-000 is done and a small change can be built, tested and reviewed by Codex.

Step 4: Refine only the next slice

Do not fully specify the entire backlog. Take the next slice's stories through the Definition of Ready, link the design, settle its edge cases and order the work. Leave later stories lighter so learning can change them cheaply.

Exit condition: the next two or three stories are Ready, ordered and independently verifiable.

Step 5: Build, inspect and adapt

Give Codex one ready story at a time. Review the plan for risky work, review the diff, run tests, use the build on a device, and update the backlog or rules with what was learned. A usable increment matters more than the number of completed tickets.

Exit condition: the slice works end to end and produces evidence for the next product decision.

Step 6: Test recovery before polishing acquisition

Use the app for several ordinary days, deliberately leave a gap, and then test catch-up. Invite a small external cohort only after data integrity and deletion behaviour are dependable.

Exit condition: users can return without loss, confusion or shame; core records remain trustworthy.

Step 7: Decide whether MVP has earned Feature 1

Only begin photo and AI work if manual logging, correction and catch-up are stable and users can explain the record. Define AI quality, uncertainty and correction measures before choosing a model or service.

Exit condition: a separate Feature 1 brief has a clear problem, privacy model, evaluation set and failure handling.

20. Working cadence for a solo product owner using Codex

Use a lightweight weekly loop rather than adopting process for its own sake.

- **Choose:** one weekly outcome and the next two or three ready stories.
- **Clarify:** settle product questions before implementation or label a time-boxed technical spike.
- **Build:** give Codex one scoped task with context, constraints and done conditions.
- **Verify:** run automated checks, review the diff and use the feature.
- **Learn:** record surprises, usability issues and decisions.

- **Reorder:** update the backlog based on evidence, not sunk cost.

Maintain one ordered backlog and one decision log. Avoid parallel lists in chat, documents and code that drift apart.

21. Sources and rationale

This pack uses the following guidance as inputs rather than as a rigid delivery methodology:

- The [Scrum Guide](#) defines a Product Goal, an emergent ordered Product Backlog, ongoing refinement and a shared Definition of Done. It stresses usable increments, inspection and adaptation rather than a fixed up-front specification.
- [Atlassian's user-story guidance](#) frames stories as concise user outcomes supported by conversation and testable confirmation/acceptance criteria.
- SVPG recommends addressing value, usability, feasibility and viability risks early and validating useful, usable and feasible solutions rather than using engineering as an expensive prototype; see [The Four Big Risks](#) and [Product Discovery](#).
- [Apple's HealthKit guidance](#) requires contextual permission, clear purpose, privacy protection and system-managed sharing controls.
- [Apple's App Review Guidelines](#) treat health data as especially sensitive, prohibit its advertising or marketing use, require accurate health claims and require clear privacy disclosure.
- [Apple's iOS guidance](#) describes Live Activities as glanceable, current Lock Screen and Dynamic Island information for ongoing events.
- [OpenAI's Codex best-practices guidance](#) recommends prompts that state goal, context, constraints and “done when”; reusable repository instructions in AGENTS.md; and a loop of tests, checks and diff review.

Appendix A: Readiness checklist for the first Codex implementation

- ☐ Product Goal accepted.
- ☐ MVP in-scope and out-of-scope accepted.
- ☐ Eight decisions in section 15 recorded.
- ☐ Minimum iOS version recorded.
- ☐ Repository created with AGENTS.md.
- ☐ Build, unit-test and UI-test commands documented and run once.
- ☐ Persistence and clock abstractions selected.

- ☐ Storyboard screens linked from the repository.
- ☐ OW-002 passes Definition of Ready.
- ☐ OW-101 passes Definition of Ready.
- ☐ A physical iPhone is available for later HealthKit and Live Activity checks.
- ☐ A decision log and one ordered backlog are the source of truth.

Appendix B: Immediate next session

The next product session should take 45 to 60 minutes:

1. Accept or amend the eight decisions in section 15.
2. Confirm the first internal-build success test: “I can choose 12 or 16 hours, start a fast, close the app, reopen it and end the same fast.”
3. Choose the minimum iOS version and local persistence approach.
4. Create the repository and add the Markdown files supplied with this pack.
5. Give OW-000 to Codex as a planning task.
6. After reviewing that plan, implement OW-002 and OW-101 as the first user-facing increment.